

Universidade de Brasília
Tópicos Avançados em Computadores

Write-Up

TryHackMe – Hash Cracking

Plataforma: TryHackMe
Dificuldade: Fácil / Médio
Data de Resolução: 10/05/2026

Autor: Mateus Reis Bastos
Matrícula: 241008513

Universidade de Brasília – UnB
2026

Sumário

1	Visão Geral	2
2	Level 1	2
2.1	Hash 1: 48bb6e862e54f2a795ffc4e541caed4d	2
2.2	Hash 2: CBFDAC6008F9CAB4083784CBD1874F76618D2A97	3
2.3	Hash 3: 1C8BFE8F801D79745C4631D09FFF36C82AA37FC4CCE4FC946683D7B336B63032	3
2.4	Hash 4: \$2y\$12\$Dwt1BZj6pcyc3Dy1FWZ5ieeUznr71EeNkJkUlypTsgbX1H68wsRom	4
2.5	Hash 5: 279412f945939ba78ce0758d3fd83daa	5
3	Level 2	6
3.1	Hash 1: F09EDCB1FCEFC6DFB23DC3505A882655FF77375ED8AA2D1C13F640FCCC2DOC85	6
3.2	Hash 2: 1DFECA0C002AE40B8619ECF94819CC1B	6
3.3	Hash 3: \$6\$aReallyHardSalt\$6WKUTqz	7
3.4	Hash 4: e5d8870e5bdd26602cab8dbe07a942c8669e56d6:tryhackme	8
4	Salas OFFSECFIL e SAWCTF	8
4.1	Hash 1: 482c811da5d5b4bc6d497ffa98491e38	8
4.2	Hash 2: 861c4f67e887dec85292d36ab05cd7a1a7275228	9
4.3	Hash 3: 4149c5cc4c378444d116d65ad5ba4099	9
4.4	Hash 4: cdeb746ec095149627348b61d4140fc58b745875	10
4.5	Hash 5: 362fda2183b7ac73400a83f6ab2c359451e48adf6c3d46a2963ee2abdf852912	11
5	Conclusão	11

1 Visão Geral

Este documento detalha o processo de identificação e quebra de *hashes* criptográficos abordados em múltiplos desafios da plataforma TryHackMe (níveis 1, 2 e salas adicionais como OFFSECFCIL e SAWCTF). O foco principal consistiu na utilização de ferramentas como HashID e Hash Analyzer para o reconhecimento de assinaturas, aliados ao uso extensivo do Hashcat e técnicas de filtro de *wordlists* para condução de ataques de força bruta. Em casos de exaustão de dicionários locais, recorreu-se a serviços de *rainbow tables* online como o CrackStation.

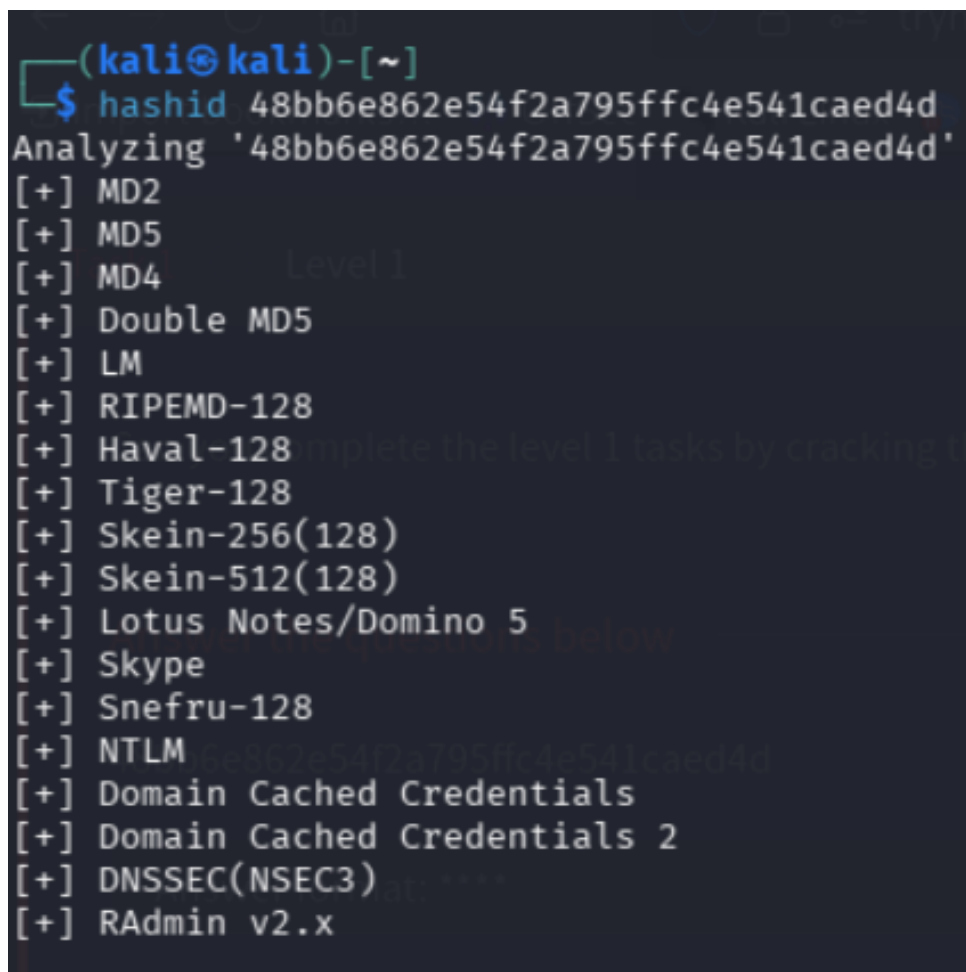
2 Level 1

2.1 Hash 1: 48bb6e862e54f2a795ffc4e541caed4d

Primeiramente, adicionamos o *hash* a um arquivo de texto (.txt) para facilitar os testes e as tentativas de quebra via terminal.

```
1 echo "48bb6e862e54f2a795ffc4e541caed4d" > hash1.txt
```

A primeira possibilidade levantada foi de que se tratava de um MD2, porém a tentativa utilizando a ferramenta **John the Ripper** não obteve sucesso.



```
(kali㉿kali)-[~]  
$ hashid 48bb6e862e54f2a795ffc4e541caed4d  
Analyzing '48bb6e862e54f2a795ffc4e541caed4d'  
[+] MD2  
[+] MD5  
[+] MD4  
[+] Double MD5  
[+] LM  
[+] RIPEMD-128  
[+] Haval-128  
[+] Tiger-128  
[+] Skein-256(128)  
[+] Skein-512(128)  
[+] Lotus Notes/Domino 5  
[+] Skype  
[+] Snefru-128  
[+] NTLM  
[+] Domain Cached Credentials  
[+] Domain Cached Credentials 2  
[+] DNSSEC(NSEC3)  
[+] RAdmin v2.x
```

Figura 1: Tentativa de quebra frustrada assumindo formato MD2.

Em seguida, ajustamos a estratégia para testar a assinatura como MD5 utilizando o **Hashcat**, o que resultou na descoberta bem-sucedida da senha.

```
(kali@kali)-[~]
$ hashcat -m 0 -a 0 hash1.txt /usr/share/wordlists/rockyou.txt
hashcat (v7.1.2) starting

OpenCL API (OpenCL 3.0 PoCL 6.0+debian Linux, None+Asserts, RELOC, SPIR-V, LLVM 18.1.8, SLEEF, DISTRO, POCL_DEBUG) - Platform #1 [The pocl project]

* Device #01: cpu-haswell-Intel(R) Core(TM) i5-10300H CPU @ 2.50GHz, 2852/5704 MB (1024 MB allocatable), 4MCU

Minimum password length supported by kernel: 0
Maximum password length supported by kernel: 256

INFO: All hashes found as potfile and/or empty entries! Use --show to display them.
For more information, see https://hashcat.net/faq/potfile

Started: Sun May 3 17:43:26 2026
Stopped: Sun May 3 17:43:26 2026

(kali@kali)-[~]
$ hashcat -m 0 -a 0 hash1.txt /usr/share/wordlists/rockyou.txt --show
```

Figura 2: Sucesso na quebra do MD5 com o Hashcat.

2.2 Hash 2: CBFDAC6008F9CAB4083784CBD1874F76618D2A97

Utilizando o **HashID**, identificamos que a assinatura correspondia a um SHA-1. A quebra foi realizada através do Hashcat no modo 100 (-m 100), aplicando um ataque de força bruta atrelado à *wordlist* RockYou.

```
(kali@kali)-[~]
$ hashid CBFDAC6008F9CAB4083784CBD1874F76618D2A97
Analyzing 'CBFDAC6008F9CAB4083784CBD1874F76618D2A97'
[+] SHA-1 0 md5 874303206306a4097a03d1633f5e
[+] Double SHA-1 md5($pass.$salt) 01dfa66e5d4d90d9892622325995
[+] RIPEMD-160 md5($salt.$pass) f0da58630310a6dd91a7d810a4c
[+] Haval-160 md5(utf16le($pass.$salt)) b31d032c1dc147a399990a71e43c
[+] Tiger-160 md5($salt.utf16le($pass)) d63d0e211dc05f618d55ef308c54
[+] HAS-160 80 HMAC-MD5 (key = $pass) fc741db0a2968c39d9c2a5cc75b0
[+] LinkedIn 80 HMAC-MD5 (key = $salt) b1d280436f45fa38eaaacac3b0051
[+] Skein-256(160) (utf16le($pass)) 2303b15bfa48c74a74758135a0d
[+] Skein-512(160) b89eaaac7e61417341b710b72776
```

Figura 3: Identificação do tipo de hash como SHA-1.

```
(kali@kali)-[~]
$ hashcat -m 100 -a 0 CBFDAC6008F9CAB4083784CBD1874F76618D2A97 /usr/share/wordlists/rockyou.txt --show
cbfdac6008f9cab4083784cbd1874f76618d2a97:password123
```

Figura 4: Quebra do SHA-1 utilizando o modo 100 do Hashcat.

2.3 Hash 3: 1C8BFE8F801D79745C4631D09FFF36C82AA37FC4CCE4FC946683D7B336B63032

Inicialmente, submetemos a *string* ao HashID, que apontou o algoritmo **Snefru-256** como a primeira opção. No entanto, o teste da string na ferramenta online CyberChef não retornou nenhum resultado válido.

```
(kali@kali)-[~]
$ hashid 1C8BFE8F801D79745C4631D09FFF36C82AA37FC4CCE4FC946683D7B336B63032
Analyzing '1C8BFE8F801D79745C4631D09FFF36C82AA37FC4CCE4FC946683D7B336B63032'
[+] Snefru-256
[+] SHA-256
[+] RIPEMD-256
[+] Haval-256
[+] GOST R 34.11-94
[+] GOST CryptoPro S-Box
[+] SHA3-256
[+] Skein-256
[+] Skein-512(256)
```

Figura 5: Resultado do HashID indicando Snefru-256 e SHA-256.

Partindo para a segunda alternativa indicada ("SHA-256"), o Hashcat foi configurado para operar no modo 1400. A injeção da *wordlist* RockYou foi suficiente para revelar o conteúdo oculto.

```
(kali@kali)-[~]
$ hashcat -m 1400 -a 0 1C8BFE8F801D79745C4631D09FFF36C82AA37FC4CCE4FC946683D7B336B63032 /usr/share/wordlists/rockyou.txt --show
1c8bfe8f801d79745c4631d09fff36c82aa37fc4cce4fc946683d7b336b63032:letmein
```

Figura 6: Sucesso na quebra do SHA-256 (modo 1400).

2.4 Hash 4: \$2y\$12\$Dwt1BZj6pcyc3Dy1FWZ5ieeUznr71EeNkJkUlypTsgbX1H68wsRom

O utilitário HashID não foi capaz de identificar este formato corretamente. Ao consultar a plataforma online **Hash Analyzer**, constatou-se que se tratava de uma assinatura **bcrypt**.

Devido ao alto custo computacional inerente ao bcrypt (algoritmo lento por design), a utilização da *wordlist* padrão do RockYou resultaria em um tempo inviável de processamento no Hashcat. Para otimizar o processo, utilizou-se o comando **grep** a fim de criar um dicionário reduzido contendo exclusivamente palavras de exatamente 4 letras.

```
1 grep -x '[a-z]{4}' /usr/share/wordlists/rockyou.txt >
   rockyou_4.txt
```

```
(kali@kali)-[~]
$ grep -x '[a-z]{4}' /usr/share/wordlists/rockyou.txt > rockyou_4letras.txt
```

Figura 7: Utilização do **grep** para filtrar a *wordlist*.

Com a lista altamente restrita, a execução do Hashcat no modo 3200 finalizou rapidamente, retornando a senha correta.

```
(kali@kali)-[~]
$ hashcat -m 3200 -a 0 '$2y$12$Dwt1BZj6pcyc3Dy1FWZ5ieeUznr71EeNkJkUlypTsgbX1H68wsRom' rockyou_4letras.txt
hashcat (v7.1.2) starting

OpenCL API (OpenCL 3.0 PoCL 6.0+debian Linux, None+Asserts, RELoc, SPIR-V, LLVM 18.1.8, SLEEP, DISTRO, POCL_DEBUG) - Platform #1 [The pocl project]

* Device #01: cpu-haswell-Intel(R) Core(TM) i5-10300H CPU @ 2.50GHz, 2852/5704 MB (1024 MB allocatable), 4MCU

Minimum password length supported by kernel: 0
Maximum password length supported by kernel: 72
Minimum salt length supported by kernel: 0
Maximum salt length supported by kernel: 256

INFO: All hashes found as potfile and/or empty entries! Use --show to display them.
For more information, see https://hashcat.net/faq/potfile

Started: Sat May 9 14:08:11 2026
Stopped: Sat May 9 14:08:11 2026

(kali@kali)-[~]
$ hashcat -m 3200 -a 0 '$2y$12$Dwt1BZj6pcyc3Dy1FWZ5ieeUznr71EeNkJkUlypTsgbX1H68wsRom' rockyou_4letras.txt --show
```

Figura 8: Quebra rápida do bcrypt com dicionário filtrado.

2.5 Hash 5: 279412f945939ba78ce0758d3fd83daa

A ferramenta HashID classificou-o primariamente como MD5. Todavia, a tentativa de quebra local exauriu o dicionário do Hashcat sem encontrar *matches* (*exhausted*). Testes assumindo o formato MD4 também retornaram o mesmo erro de exaustão.

Considerando que o problema residia nas limitações do dicionário local, o *hash* foi submetido ao serviço **CrackStation**, o qual retornou o texto em claro com êxito graças à sua vasta *rainbow table* em nuvem.

```
(kali㉿kali)-[~]
└─$ hashid 279412f945939ba78ce0758d3fd83daa
Analyzing '279412f945939ba78ce0758d3fd83daa'
[+] MD2
[+] MD5 2410 Cisco-ASA MD5
[+] MD4 2500 WPA-EAPOL-PBKDF2 1
[+] Double MD5 2501 WPA-EAPOL-PMK 14
[+] LM 2600 md5(md5($pass))
[+] RIPEMD-128 2630 md5(md5($pass.$salt)) *
[+] Haval-128 3000 LM
[+] Tiger-128 3100 Oracle H: Type (Oracle 7+)
[+] Skein-256(128) 3200 bcrypt $2*$, Blowfish (Unix)
[+] Skein-512(128) 3500 md5(md5(md5($pass)))
[+] Lotus Notes/Domino 5 3610 md5(md5(md5($pass)).$salt) *
[+] Skype 3710 md5($salt.md5($pass))
[+] Snefru-128
[+] NTLM
[+] Domain Cached Credentials
[+] Domain Cached Credentials 2
[+] DNSSEC(NSEC3)
[+] RAdmin v2.x
```

Figura 9: Identificação inconclusiva/exaustão de *wordlist* local.



Figura 10: Sucesso na recuperação da senha via CrackStation.

3 Level 2

3.1 Hash 1: F09EDCB1FCEFC6DFB23DC3505A882655FF77375ED8AA2D1C13F640FCCC2D0C85

As opções sugeridas pelo HashID apontavam tanto para Snefru-256 quanto para SHA-256. Como as instruções da fase exigiam priorizar o uso do Hashcat, o formato **SHA-256** foi escolhido como vetor de teste inicial, sendo quebrado instantaneamente.

```
(kali@kali)~$ hashid F09EDCB1FCEFC6DFB23DC3505A882655FF77375ED8AA2D1C13F640FCCC2D0C85
Analyzing 'F09EDCB1FCEFC6DFB23DC3505A882655FF77375ED8AA2D1C13F640FCCC2D0C85'
[+] Snefru-256
[+] SHA-256
[+] RIPEMD-256
[+] Haval-256
[+] GOST R 34.11-94
[+] GOST CryptoPro S-Box
[+] SHA3-256
[+] Skein-256
[+] Skein-512(256)
```

Figura 11: Sondagem dos formatos via HashID.

```
(kali@kali)~$ hashcat -m 1400 -a 0 F09EDCB1FCEFC6DFB23DC3505A882655FF77375ED8AA2D1C13F640FCCC2D0C85 /usr/share/wordlists/rockyou.txt
hashcat (v7.1.2) starting

OpenCL API (OpenCL 3.0 PoCL 6.0+debian Linux, None+Asserts, RELOC, SPIR-V, LLVM 18.1.8, SLEEF, DISTRO, POCL_DEBUG) - Platform #1 [The pocl project]

* Device #01: cpu-haswell-Intel(R) Core(TM) i5-10300H CPU @ 2.50GHz, 2852/5704 MB (1024 MB allocatable), 4MCU

Minimum password length supported by kernel: 0
Maximum password length supported by kernel: 256

Hashes: 1 digests; 1 unique digests, 1 unique salts
Bitmaps: 16 bits, 65536 entries, 0x0000ffff mask, 262144 bytes, 5/13 rotates
Rules: 1

Optimizers applied:
* Zero-Byte
* Early-Skip
* Not-Salted
* Not-Iterated
* Single-Hash
* Single-Salt
* Raw-Hash
```

Figura 12: Confirmação da senha usando o Hashcat em SHA-256.

3.2 Hash 2: 1DFECA0C002AE40B8619ECF94819CC1B

As tentativas sistemáticas no Hashcat utilizando os modos referentes a MD2, MD4 e MD5 esgotaram completamente a base de senhas (*exhausted*). Ao desviar a abordagem e submetê-lo ao banco de dados do CrackStation, a senha foi revelada e a assinatura foi confirmada como sendo do tipo **NTLM**.

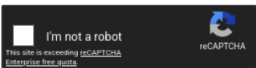
```
(kali㉿kali)-[~]
$ hashid 1DFECA0C002AE40B8619ECF94819CC1B
Analyzing '1DFECA0C002AE40B8619ECF94819CC1B'
[+] MD2
[+] MD5
[+] MD4
[+] Double MD5
[+] LM
[+] RIPEMD-128
[+] Haval-128
[+] Tiger-128
[+] Skein-256(128)
[+] Skein-512(128)
[+] Lotus Notes/Domino 5
[+] Skype
[+] Snefru-128
[+] NTLM
[+] Domain Cached Credentials
[+] Domain Cached Credentials 2
[+] DNSSEC(NSEC3)
[+] RAdmin v2.x
```

Figura 13: Sondagem dos formatos via HashID.

Free Password Hash Cracker

Enter up to 20 non-salted hashes, one per line:

1DFECA0C002AE40B8619ECF94819CC1B



Crack Hashes

Supports: LM, NTLM, md2, md4, md5, md5(md5_hex), md5-hall, sha1, sha224, sha256, sha384, sha512, rpeMD160, whirlpool, MySQL 4.1+ (sha1(sha1_bin)), QubesV3.1BackupDefaults

Hash	Type	Result
1DFECA0C002AE40B8619ECF94819CC1B	NTLM	n63umy8lkf4i

Color Codes: Green Exact match, Yellow Partial match, Red Not found.

Figura 14: Sucesso na recuperação da senha via CrackStation.

3.3 Hash 3: \$6\$aReallyHardSalt\$6WKUTqz...

A inspeção inicial com o HashID reconheceu a formatação clássica do **SHA-512 Crypt** (identificada pelo prefixo \$6\$). Com isso, o Hashcat foi configurado para o modo 1800, utilizando força bruta com o dicionário RockYou para processar o alvo e extrair a *flag*.

```
(kali㉿kali)-[~]
$ hashid '$6$aReallyHardSalt$6WKUTqz.UQQmrm0p/T7MPpMbGnNzXPMAXi4bJm19be.cfi3/qxIf.hsGpS41BqMhSrHVXgMpdjS6xeKZAs02.'
Analyzing '$6$aReallyHardSalt$6WKUTqz.UQQmrm0p/T7MPpMbGnNzXPMAXi4bJm19be.cfi3/qxIf.hsGpS41BqMhSrHVXgMpdjS6xeKZAs02.'
[+] SHA-512 Crypt
```

Figura 15: Identificação exata do SHA-512 Crypt pelo prefixo.



Figura 16: Operação bem-sucedida do Hashcat no modo 1800.

3.4 Hash 4: e5d8870e5bdd26602cab8dbe07a942c8669e56d6:tryhackme

A estrutura <hash>:<salt> e as dicas intrínsecas da plataforma confirmaram a natureza de um **HMAC-SHA1**. Passando essa matriz diretamente para o Hashcat rodando no modo específico 160, a correspondência foi encontrada.

4 Salas OFFSECFIL e SAWCTF

4.1 Hash 1: 482c811da5d5b4bc6d497ffa98491e38

Dada a rapidez do serviço externo, optou-se pela verificação direta no CrackStation, que identificou e revelou a senha em claro.

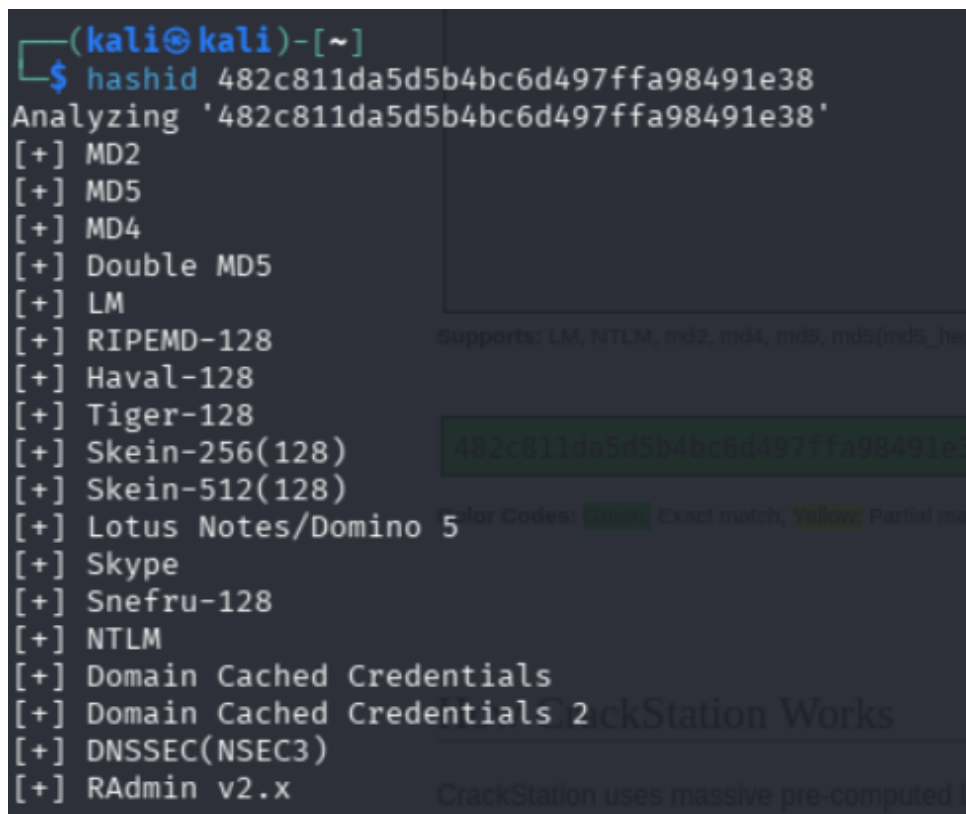
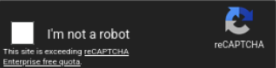


Figura 17: Sondagem dos formatos via HashID.

Free Password Hash Cracker

Enter up to 20 non-salted hashes, one per line:

482c811da5d5b4bc6d497ffa98491e38



[Crack Hashes](#)

Supports: LM, NTLM, md2, md4, md5, md5(md5_hex), md5-half, sha1, sha224, sha256, sha384, sha512, rpeMD160, whirlpool, MySQL 4.1+ (sha1(sha1_bin)), QubesV3.1BackupDefaults

Hash	Type	Result
482c811da5d5b4bc6d497ffa98491e38	md5	password123

Color Codes: Green Exact match, Yellow Partial match, Red Not found.

[Download CrackStation's Wordlist](#)

Figura 18: Sucesso na recuperação da senha via CrackStation.

4.2 Hash 2: 861c4f67e887dec85292d36ab05cd7a1a7275228

Similar ao desafio anterior, o *hash* foi alimentado no portal CrackStation e quebrado com sucesso imediatamente.

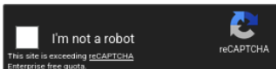
```
(kali㉿kali)-[~]
└─$ hashid 861c4f67e887dec85292d36ab05cd7a1a7275228
Analyzing '861c4f67e887dec85292d36ab05cd7a1a7275228'
[+] SHA-1
[+] Double SHA-1
[+] RIPEMD-160
[+] Haval-160
[+] Tiger-160
[+] HAS-160
[+] LinkedIn
[+] Skein-256(160)
[+] Skein-512(160)
```

Figura 19: Sondagem dos formatos via HashID.

Free Password Hash Cracker

Enter up to 20 non-salted hashes, one per line:

861c4f67e887dec85292d36ab05cd7a1a7275228



[Crack Hashes](#)

Supports: LM, NTLM, md2, md4, md5, md5(md5_hex), md5-half, sha1, sha224, sha256, sha384, sha512, rpeMD160, whirlpool, MySQL 4.1+ (sha1(sha1_bin)), QubesV3.1BackupDefaults

Hash	Type	Result
861c4f67e887dec85292d36ab05cd7a1a7275228	sha1	easy

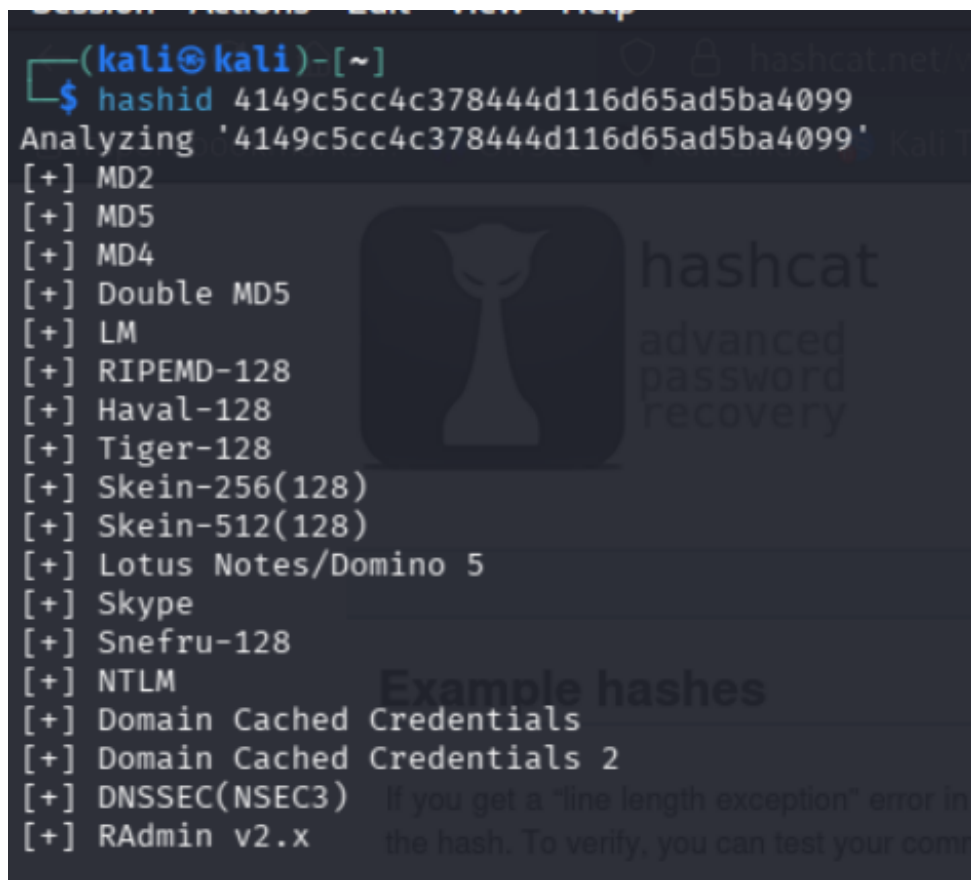
Color Codes: Green Exact match, Yellow Partial match, Red Not found.

Figura 20: Sucesso na recuperação da senha via CrackStation.

4.3 Hash 3: 4149c5cc4c378444d116d65ad5ba4099

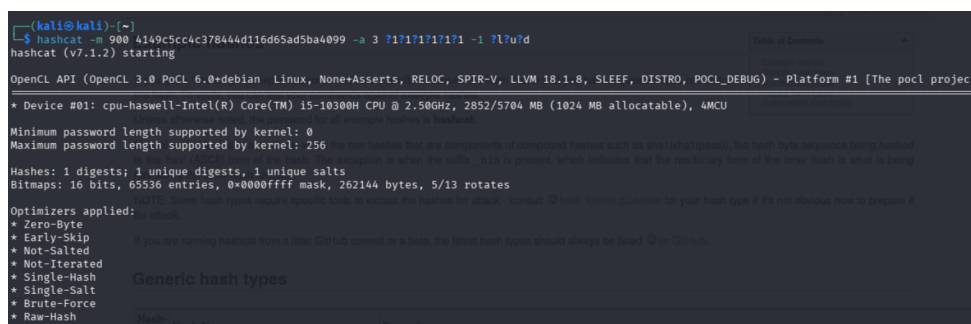
Após falhas iniciais assumindo o comportamento de um formato MD5, a alteração dos parâmetros do Hashcat para focar na matriz **MD4** (modo 900) e a execução de força

bruta normalizada entregaram a *string* verdadeira.



```
(kali@kali)-[~]
$ hashid 4149c5cc4c378444d116d65ad5ba4099
Analyzing '4149c5cc4c378444d116d65ad5ba4099'
[+] MD2
[+] MD5
[+] MD4
[+] Double MD5
[+] LM
[+] RIPEMD-128
[+] Haval-128
[+] Tiger-128
[+] Skein-256(128)
[+] Skein-512(128)
[+] Lotus Notes/Domino 5
[+] Skype
[+] Snefru-128
[+] NTLM
[+] Domain Cached Credentials
[+] Domain Cached Credentials 2
[+] DNSSEC(NSEC3)
[+] RAdmin v2.x
```

Figura 21: Classificação inicial falha e desvio para MD4.



```
(kali@kali)-[~]
$ hashcat -m 900 4149c5cc4c378444d116d65ad5ba4099 -a 3 ?1?1?1?1?1 -1 ?1?u?d
hashcat (v7.1.2) starting

OpenCL API (OpenCL 3.0 PoCL 6.0+debian Linux, None+Asserts, RELOC, SPIR-V, LLVM 18.1.8, SLEEF, DISTRO, POCL_DEBUG) - Platform #1 [The pocl project]

* Device #01: cpu-haswell-Intel(R) Core(TM) i5-10300H CPU @ 2.50GHz, 2852/5704 MB (1024 MB allocatable), 4MCU

Minimum password length supported by kernel: 0
Maximum password length supported by kernel: 256

Hashes: 1 digests; 1 unique digests, 1 unique salts
Bitmaps: 16 bits, 65536 entries, 0x0000ffff mask, 262144 bytes, 5/13 rotates

Optimizers applied:
* Zero-Byte
* Early-Skip
* Not-Salted
* Not-Iterated
* Single-Hash
* Single-Salt
* Brute-Force
* Raw-Hash

Generic hash types
MD4, MD5, MD6, SHA1, SHA256, SHA512, etc.
```

Figura 22: Quebra efetiva do formato MD4 (modo 900).

4.4 Hash 4: cdeb746ec095149627348b61d4140fc58b745875

O enunciado entregava diretamente a variável de **Salt**: **satech** combinada com as dicas do tipo de criptografia. Bastou formatar a entrada adequadamente para o Hashcat processar e resolver o desafio.

```
(kali@kali)~$ hashcat -m 160 cdeb746ec095149627348b6d4140fc58b745875:satech /usr/share/wordlists/rockyou.txt
hashcat (v7.1.2) starting job [hashcat]

OpenCL API (OpenCL 3.0 PoCL 6.0+debian Linux, None+Asserts, RELOC, SPIR-V, LLVM 18.1.8, SLEEF, DISTRO, POCL_DEBUG) - Platform #1 [The pocl project]

* Device #01: cpu-haswell-Intel(R) Core(TM) i5-10300H CPU @ 2.50GHz, 2852/5704 MB (1024 MB allocatable), 4MCU

Minimum password length supported by kernel: 0
Maximum password length supported by kernel: 256
Minimum salt length supported by kernel: 0
Maximum salt length supported by kernel: 256

Hashes: 1 digests; 1 unique digests, 1 unique salts
Bitmaps: 16 bits, 65536 entries, 0x0000ffff mask, 262144 bytes, 5/13 rotates
Rules: 1

Optimizers applied:
* Zero-Byte
* Not-Iterated
* Single-Hash
* Single-Salt
Generic hash types
```

Figura 23: Uso de salt pré-definido acoplado ao hash original.

4.5 Hash 5: 362fda2183b7ac73400a83f6ab2c359451e48adf6c3d46a2963ee2abdf852912

Para finalizar, o cenário restringia o espaço de busca (*keyspace*) informando que a senha possuía apenas letras minúsculas, números e um comprimento máximo de 6 caracteres. Essa configuração ideal permitiu definir uma máscara de ataque bruta super rápida no Hashcat.

```
hashcat -m 1400 362fda2183b7ac73400a83f6ab2c359451e48adf6c3d46a2963ee2abdf852912 -a 3 ?l?l?l?l?l?l -i ?l?d
hashcat (v7.1.2) starting

OpenCL API (OpenCL 3.0 PoCL 6.0+debian Linux, None+Asserts, RELOC, SPIR-V, LLVM 18.1.8, SLEEF, DISTRO, POCL_DEBUG) - Platform #1 [The pocl project]

* Device #01: cpu-haswell-Intel(R) Core(TM) i5-10300H CPU @ 2.50GHz, 2852/5704 MB (1024 MB allocatable), 4MCU

Minimum password length supported by kernel: 0
Maximum password length supported by kernel: 256

Hashes: 1 digests; 1 unique digests, 1 unique salts
Bitmaps: 16 bits, 65536 entries, 0x0000ffff mask, 262144 bytes, 5/13 rotates

Optimizers applied:
* Zero-Byte
* Early-Skip
* Not-Salted
* Not-Iterated
* Single-Hash
* Single-Salt
* Brute-Force
* Raw-Hash
Generic hash types
```

Figura 24: Força bruta pura aplicando regras customizadas de caracteres.

5 Conclusão

Os exercícios demonstraram a importância crítica não só da identificação prévia e correta dos algoritmos de *hashing*, mas da maleabilidade analítica durante o processo de quebra de senhas. Observou-se na prática que dicionários não são balas de prata: algoritmos com custo computacional alto (como *bcrypt*) exigem *wordlists* refinadas cirurgicamente por utilitários como o **grep**, enquanto restrições de formatação impulsionam a necessidade de ataques de máscara no Hashcat.

Ademais, quando o esforço computacional local atinge um limiar (*exhausted*), serviços como o CrackStation proveem suporte indispensável validando ou quebrando *hashes* legados do tipo NTLM e MD4 de forma extremamente eficiente.